Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра ИИТ

Лабораторная работа №2

По дисциплине «КиСБ»

Тема: «Лабораторная работа №2. Избыточное кодирование данных в информационных системах. Код Хемминга и итеративные коды»

Выполнил:
Студент 4 курса
Группы АС-66
Лысюк Р. А.
Проверила:
Хацкевич А. С.

Брест 2026

Цель работы: освоить методы помехоустойчивого кодирования двоичных данных на основе кода Хемминга с минимальным кодовым расстоянием 3 и 4, а также итеративных (двумерных) кодов с проверками на чётность, и получить практические навыки обнаружения и исправления ошибок в кодовых словах.

Ход работы

1. Закрепить теоретические знания по использованию методов помехоустойчивого кодирования для повышения надёжности передачи и хранения двоичных данных в памяти компьютера.
2. Разработать приложение для кодирования/декодирования двоичной информации кодом Хемминга с d_min = 3 и d_min = 4 (часть 1) и итеративным кодом с различной относительной избыточностью (часть 2).
3. Оформить результаты в виде описания приложения, методики экспериментов и результатов эксперимента.

Вариант 4:

4. Составить проверочные матрицы кодов Хемминга для кодирования 9-битных сообщений по схеме, представленной на рис. 5.3 (с учетом и без учета символа X_{hv}).

Текст программы:

```
#lab2_1
import sys

import random


if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")


def hamming_r(k):
    r = 1
    while (1 << r) < k + r + 1:
        r += 1
    return r


def build_h(k, r):
    n = k + r
    cols = []
    used = set()
    wt = 1
    while len(cols) < k:
```

```
        for v in range(1, 1 << r):
            if v in used:
                continue
            if bin(v).count("1") == wt:
                cols.append(v)
                used.add(v)
                if len(cols) == k:
                    break
        wt += 1
    for i in range(r):
        cols.append(1 << i)
    H = [[(cols[c] >> i) & 1 for c in range(n)] for i in range(r)]
    return H, n


def encode_h(X, H, extended):
    Xr = []
    for row in H:
        s = 0
        for j in range(len(X)):
            s ^= X[j] & row[j]
        Xr.append(s)
    Xn = X + Xr
    if extended:
        Xn = Xn + [sum(Xn) % 2]
    return Xn


def syndrome(Y, H):
    return [sum(Y[j] & row[j] for j in range(len(Y))) % 2 for row in H]


def decode_h(Y, H, k, n, extended):
    if extended:
        body = Y[:n]
        total = sum(body) % 2
        S = syndrome(body, H)
        if all(s == 0 for s in S) and total == 0:
            return Y[:k], S, None
        if all(s == 0 for s in S) and total == 1:
            # ошибка в самом проверочном бите чётности
            Y2 = Y[:]
            Y2[-1] ^= 1
```

```
            return Y2[:k], S, n
        for c in range(n):
            col = [H[i][c] for i in range(len(H))]
            if col == S:
                Y2 = Y[:]
                Y2[c] ^= 1
                return Y2[:k], S, c
        return Y[:k], S, None
    else:
        S = syndrome(Y, H)
        if all(s == 0 for s in S):
            return Y[:k], S, None
        for c in range(n):
            col = [H[i][c] for i in range(len(H))]
            if col == S:
                Y2 = Y[:]
                Y2[c] ^= 1
                return Y2[:k], S, c
        return Y[:k], S, None


def run():
    print("=== LAB 2.1. Hamming code ===")
    k = 16
    for dmin in (3, 4):
        r = hamming_r(k)
        H, n = build_h(k, r)
        extended = (dmin == 4)
        info = [random.randint(0, 1) for _ in range(k)]
        Xn = encode_h(info, H, extended)
        print(f"\nk={k} r={r} n={len(Xn)} dmin={dmin}")
        print(f"  Xk = {''.join(map(str, info))}")
        print(f"  Xn = {''.join(map(str, Xn))}")
        for err in (0, 1, 2):
            Y = Xn[:]
            pos = random.sample(range(len(Xn)), err)
            for p in pos:
                Y[p] ^= 1
            dec, S, fixed = decode_h(Y, H, k, n, extended)
            ok = dec == info
            print(f"  err={err} pos={pos} S={''.join(map(str, S))} "
```

```
            f"fixed={fixed} decoded_ok={ok}")

if __name__ == "__main__":
    random.seed(20241018)
    run()


#lab2_2
import sys
import random

if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

def build_rect(k, k1, k2):
    grid = [[0] * k2 for _ in range(k1)]
    for i in range(k):
        grid[i // k2][i % k2] = random.randint(0, 1)
    return grid

def encode_rect(grid, groups):
    rows = len(grid)
    cols = len(grid[0])
    rh = [sum(grid[i]) % 2 for i in range(rows)]
    rv = [sum(grid[i][j] for i in range(rows)) % 2 for j in range(cols)]
    rhv = sum(rh) % 2
    extra = rh + rv
    if groups >= 3:
        extra.append(rhv)
    if groups >= 4:
        d1 = [sum(grid[i][(i + t) % cols] for i in range(rows)) % 2 for t in range(cols)]
        extra += d1
    if groups >= 5:
        d2 = [sum(grid[i][(cols - 1 - i + t) % cols] for i in range(rows)) % 2 for t in range(cols)]
        extra += d2
    return grid, rh, rv, rhv, extra

def detect_rect(grid, rh, rv, rhv, groups):
    rows = len(grid)
    cols = len(grid[0])
    rh2 = [sum(grid[i]) % 2 for i in range(rows)]
```

```
    rv2 = [sum(grid[i][j] for i in range(rows)) % 2 for j in range(cols)]
    rhv2 = sum(rh2) % 2
    bad_rows = [i for i in range(rows) if rh[i] != rh2[i]]
    bad_cols = [j for j in range(cols) if rv[j] != rv2[j]]
    result = []
    if len(bad_rows) == 1 and len(bad_cols) == 1:
        result.append((bad_rows[0], bad_cols[0]))
    return result, rhv != rhv2, bad_rows, bad_cols


def fix_rect(grid, positions):
    for i, j in positions:
        grid[i][j] ^= 1


def run():
    print("=== LAB 2.2. Iterative (rectangular) code ===")
    k = 32
    for k1, k2 in [(4, 8), (2, 16), (8, 4)]:
        print(f"\nk={k}  k1={k1}  k2={k2}")
        for groups in (2, 3, 4, 5):
            grid = build_rect(k, k1, k2)
            orig = [row[:] for row in grid]
            grid, rh, rv, rhv, extra = encode_rect(grid, groups)
            for err in (1, 2, 3):
                g2 = [row[:] for row in grid]
                pos = random.sample([(i, j) for i in range(k1) for j in range(k2)], err)
                for i, j in pos:
                    g2[i][j] ^= 1
                found, rhv_bad, br, bc = detect_rect(g2, rh, rv, rhv, groups)
                fix_rect(g2, found)
                ok = g2 == orig
                print(f"  groups={groups} err={err} pos={pos} "
                      f"detected={len(found)} bad_rows={br} bad_cols={bc} decoded_ok={ok}")


if __name__ == "__main__":
    random.seed(20241018)
    run()
```

Результат работы программы:

Lab2_1

```
PS C:\Users\lysyu\OneDrive\Desktop\КиСБ\KiSB-2026-autumn\lab2\Лысюк> python lab2_1.py
=== LAB 2.1. Hamming code ===

k=16 r=5 n=21 dmin=3
  Xk = 1001001000011110
  Xn = 100100100001111011000
  err=0 pos=[] S=00000 fixed=None decoded_ok=True
  err=1 pos=[2] S=00100 fixed=2 decoded_ok=True
  err=2 pos=[0, 4] S=10001 fixed=11 decoded_ok=False

k=16 r=5 n=22 dmin=4
  Xk = 1110010010110001
  Xn = 1110010010110001111010
  err=0 pos=[] S=00000 fixed=None decoded_ok=True
  err=1 pos=[11] S=10001 fixed=11 decoded_ok=True
  err=2 pos=[11, 3] S=10011 fixed=None decoded_ok=False
```

Lab2_2

```
PS C:\Users\lysyu\OneDrive\Desktop\КиСБ\KiSB-2026-autumn\lab2\Лысюк> python lab2_2.py
k=32  k1=4  k2=8
  groups=2 err=1 pos=[(0, 1)] detected=1 bad_rows=[0] bad_cols=[1] decoded_ok=True
  groups=2 err=2 pos=[(0, 0), (2, 2)] detected=0 bad_rows=[0, 2] bad_cols=[0, 2] decoded_ok=False
  groups=2 err=3 pos=[(2, 6), (2, 7), (0, 7)] detected=1 bad_rows=[0] bad_cols=[6] decoded_ok=False
  groups=3 err=1 pos=[(1, 2)] detected=1 bad_rows=[1] bad_cols=[2] decoded_ok=True
  groups=3 err=2 pos=[(1, 1), (0, 6)] detected=0 bad_rows=[0, 1] bad_cols=[1, 6] decoded_ok=False
  groups=3 err=3 pos=[(0, 3), (1, 4), (2, 6)] detected=0 bad_rows=[0, 1, 2] bad_cols=[3, 4, 6] decoded_ok=False
  groups=4 err=1 pos=[(3, 6)] detected=1 bad_rows=[3] bad_cols=[6] decoded_ok=True
  groups=4 err=2 pos=[(0, 2), (3, 5)] detected=0 bad_rows=[0, 3] bad_cols=[2, 5] decoded_ok=False
  groups=4 err=3 pos=[(3, 4), (1, 4), (0, 5)] detected=0 bad_rows=[0, 1, 3] bad_cols=[5] decoded_ok=False
  groups=5 err=1 pos=[(3, 5)] detected=1 bad_rows=[3] bad_cols=[5] decoded_ok=True
  groups=5 err=2 pos=[(0, 1), (3, 6)] detected=0 bad_rows=[0, 3] bad_cols=[1, 6] decoded_ok=False
  groups=5 err=3 pos=[(3, 7), (0, 6), (3, 4)] detected=0 bad_rows=[0] bad_cols=[4, 6, 7] decoded_ok=False

k=32  k1=2  k2=16
  groups=2 err=1 pos=[(1, 15)] detected=1 bad_rows=[1] bad_cols=[15] decoded_ok=True
  groups=2 err=2 pos=[(1, 10), (0, 12)] detected=0 bad_rows=[0, 1] bad_cols=[10, 12] decoded_ok=False
  groups=2 err=3 pos=[(0, 6), (0, 7), (0, 3)] detected=0 bad_rows=[0] bad_cols=[3, 6, 7] decoded_ok=False
  groups=3 err=1 pos=[(1, 4)] detected=1 bad_rows=[1] bad_cols=[4] decoded_ok=True
  groups=3 err=2 pos=[(0, 3), (1, 7)] detected=0 bad_rows=[0, 1] bad_cols=[3, 7] decoded_ok=False
  groups=3 err=3 pos=[(0, 15), (0, 6), (1, 9)] detected=0 bad_rows=[1] bad_cols=[6, 9, 15] decoded_ok=False
  groups=4 err=1 pos=[(1, 12)] detected=1 bad_rows=[1] bad_cols=[12] decoded_ok=True
  groups=4 err=2 pos=[(1, 5), (0, 5)] detected=0 bad_rows=[0, 1] bad_cols=[] decoded_ok=False
  groups=4 err=3 pos=[(0, 14), (0, 13), (1, 2)] detected=0 bad_rows=[1] bad_cols=[2, 13, 14] decoded_ok=False
  groups=5 err=1 pos=[(0, 3)] detected=1 bad_rows=[0] bad_cols=[3] decoded_ok=True
  groups=5 err=2 pos=[(1, 11), (0, 14)] detected=0 bad_rows=[0, 1] bad_cols=[11, 14] decoded_ok=False
  groups=5 err=3 pos=[(1, 12), (1, 14), (1, 7)] detected=0 bad_rows=[1] bad_cols=[7, 12, 14] decoded_ok=False

k=32  k1=8  k2=4
  groups=2 err=1 pos=[(5, 0)] detected=1 bad_rows=[5] bad_cols=[0] decoded_ok=True
  groups=2 err=2 pos=[(2, 3), (5, 2)] detected=0 bad_rows=[2, 5] bad_cols=[2, 3] decoded_ok=False
  groups=2 err=3 pos=[(0, 2), (4, 2), (0, 3)] detected=1 bad_rows=[4] bad_cols=[3] decoded_ok=False
  groups=3 err=1 pos=[(7, 1)] detected=1 bad_rows=[7] bad_cols=[1] decoded_ok=True
  groups=3 err=2 pos=[(2, 1), (1, 3)] detected=0 bad_rows=[1, 2] bad_cols=[1, 3] decoded_ok=False
  groups=3 err=3 pos=[(2, 2), (5, 3), (5, 1)] detected=0 bad_rows=[2] bad_cols=[1, 2, 3] decoded_ok=False
  groups=4 err=1 pos=[(1, 3)] detected=1 bad_rows=[1] bad_cols=[3] decoded_ok=True
  groups=4 err=2 pos=[(4, 3), (3, 3)] detected=0 bad_rows=[3, 4] bad_cols=[] decoded_ok=False
  groups=4 err=3 pos=[(1, 0), (4, 2), (7, 1)] detected=0 bad_rows=[1, 4, 7] bad_cols=[0, 1, 2] decoded_ok=False
  groups=5 err=1 pos=[(7, 0)] detected=1 bad_rows=[7] bad_cols=[0] decoded_ok=True
  groups=5 err=2 pos=[(1, 0), (7, 3)] detected=0 bad_rows=[1, 7] bad_cols=[0, 3] decoded_ok=False
  groups=5 err=3 pos=[(6, 0), (6, 3), (1, 1)] detected=0 bad_rows=[1] bad_cols=[0, 1, 3] decoded_ok=False
PS C:\Users\lysyu\OneDrive\Desktop\КиСБ\KiSB-2026-autumn\lab2\Лысюк>
```

Вывод: В ходе работы реализованы код Хемминга с d_min = 3 и d_min = 4 и итеративный код с проверками по строкам, столбцам и диагоналям. Код Хемминга исправляет все одиночные ошибки; при d_min = 3 двойная ошибка обнаруживается, но может быть принята за одиночную, а при d_min = 4 (расширенный код с общим битом чётности) двойная ошибка гарантированно обнаруживается без порчи слова. Итеративный код уверенно исправляет одиночные ошибки при любых размерах матрицы, однако двойные ошибки в вершинах прямоугольника не обнаруживаются — это ограничение двумерного кода с d_min = 4. Рост числа групп паритетов повышает обнаруживающую способность, но для исправления кратных ошибок требуется многомерная структура с независимыми проверками по плоскостям.